

Manuel technique — Système d'identification IA (Vision) Pièces

Plateforme : Pièces — Marketplace de pièces auto d'occasion **Marché :** Côte d'Ivoire (Abidjan) **Devise :** FCFA **Version :** 1.0 — Mars 2026

Table des matières

1. Vue d'ensemble du système IA
2. Architecture technique
3. Intégration Google Gemini
4. Prompt d'identification
5. Configuration et variables d'environnement
6. Endpoint : identification par photo
7. Upload de photo : formats et limites
8. Les 3 niveaux de confiance
9. Confiance haute ($\geq 70\%$) : identifié
10. Confiance moyenne (30-70%) : désambiguïsation
11. Confiance basse ($< 30\%$) : échec
12. Endpoint : désambiguïsation
13. Correspondance catalogue (matching)
14. Décodage VIN (NHTSA VPIC)
15. Recherche textuelle et synonymes
16. Navigation par marque / modèle / année
17. Catégories de pièces
18. Compatibilité véhicule
19. Traitement d'images asynchrone (queue)
20. Variante d'images et qualité
21. Évaluation de qualité photo
22. Job CATALOG_AI_IDENTIFY
23. Stockage R2 (Cloudflare)
24. Upload catalogue vendeur
25. Détection bait-and-switch
26. Intégration WhatsApp (photo par message)

- 27. Rate limiting et quotas
- 28. Gestion d'erreurs et fallbacks
- 29. Tests unitaires
- 30. Modèles de données Prisma
- 31. Validation des données (Zod)
- 32. État de l'implémentation
- 33. Référence des fichiers
- 34. FAQ technique

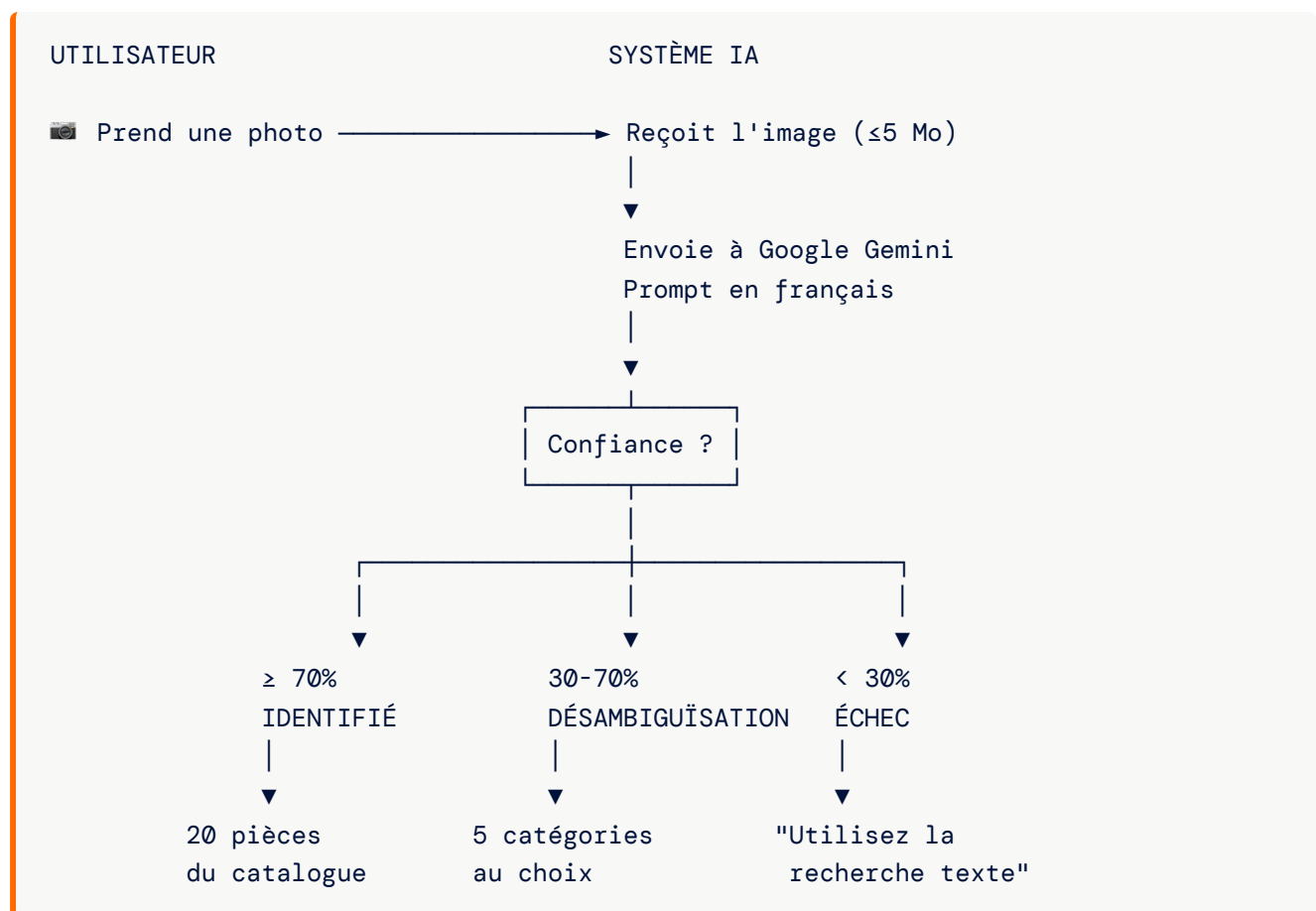
1. Vue d'ensemble du système IA

Le système d'identification IA de Pièces permet aux utilisateurs de **photographier une pièce auto usagée** et d'obtenir automatiquement son nom, sa catégorie et les pièces correspondantes dans le catalogue.

Les 5 composants

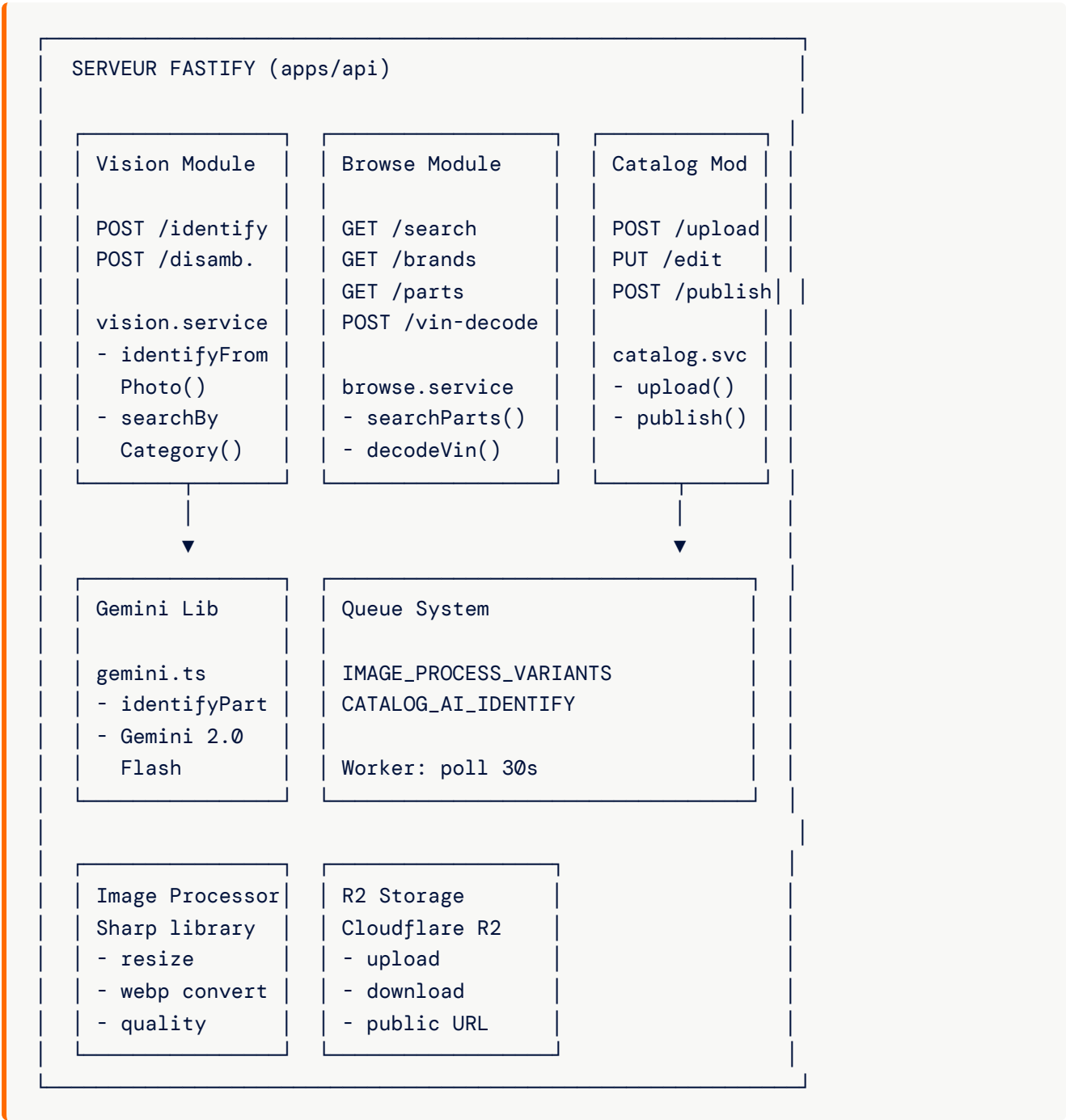
SYSTÈME D'IDENTIFICATION IA PIÈCES		
1. VISION	2. RECHERCHE	3. VIN
Photo → IA	Texte → Catalogue	VIN → Véhicule
Gemini 2.0 Flash	Synonymes + Trigram	NHTSA VPIC
4. TRAITEMENT	5. CATALOGUE	
Queue async	Upload vendeur	
Sharp variants	AI pré-remplissage	
Qualité photo	Publish workflow	

Flux utilisateur principal



2. Architecture technique

2.1 Composants



2.2 Flux de données

Identification en temps réel (synchrone) :

Photo → Vision Route → Gemini API → Matching catalogue → Résultat

Traitement vendeur (asynchrone) :

Upload vendeur → R2 → Queue → Sharp (variantes) + Gemini (identification)
→ Mise à jour CatalogItem

3. Intégration Google Gemini

3.1 SDK utilisé

```
import { GoogleGenerativeAI } from '@google/generative-ai';
```

3.2 Modèle

Paramètre	Valeur
Modèle par défaut	gemini-2.0-flash
Variable de config	GEMINI_MODEL
Initialisation	Lazy singleton

3.3 Processus d'appel

```
// 1. Encoder l'image en base64
const base64Image = imageBuffer.toString('base64');

// 2. Appeler Gemini avec le prompt + l'image
const result = await model.generateContent([
  PROMPT,
  {
    inlineData: {
      mimeType: 'image/jpeg', // ou png, webp
      data: base64Image
    }
  }
]);

// 3. Extraire la réponse texte
const text = result.response.text();

// 4. Nettoyer le markdown (``json ... ``)
const cleaned = text.replace(/``json\n?/g, '').replace(/``\n?/g, '');

// 5. Parser le JSON
const identification = JSON.parse(cleaned);

// 6. Normaliser la confiance (0-1)
if (identification.confidence > 1) {
  identification.confidence = identification.confidence / 100;
}
```

3.4 Suivi des quotas

```
// Compteur d'appels (estimation)
let callCount = 0;
const QUOTA_ALERT_THRESHOLD = 0.8; // 80% de 100 appels estimés

// Alerte si quota proche
if (callCount / 100 >= QUOTA_ALERT_THRESHOLD) {
  logger.warn({ callCount }, 'GEMINI_QUOTA_ALERT');
}
```

3.5 Gestion des erreurs Gemini

Erreur	Comportement
Clé API manquante	Log warning, retourne <code>null</code>
API en erreur	Log warning + callCount, retourne <code>null</code>
JSON invalide	Log warning, retourne <code>null</code>
Toutes les erreurs	Fallback vers identification manuelle

4. Prompt d'identification

4.1 Prompt complet

```
Analyze this auto part image from an Ivory Coast (Côte d'Ivoire) marketplace.
Return ONLY a valid JSON object with these fields:
{
  "name": "Part name in French",
  "category": "One of: Filtration, Freinage, Suspension, Moteur, Transmission,
              Electricité, Carrosserie, Echappement, Refroidissement, Autre",
  "oemReference": "OEM reference if visible on the part, null otherwise",
  "vehicleCompatibility": "Suggested vehicle compatibility if identifiable
                          (e.g. 'Toyota Hilux 2005-2015'), null otherwise",
  "suggestedPrice": "Estimated price in FCFA for Abidjan market, null if unknown",
  "confidence": "Number between 0 and 1 indicating identification confidence"
}
Only return valid JSON, no markdown, no other text.
```

4.2 Structure de la réponse attendue

```
interface PartIdentification {
  name: string;           // Nom en français
  category: string;       // Catégorie normalisée
  oemReference: string | null; // Référence OEM si visible
  vehicleCompatibility: string | null; // Compatibilité véhicule
  suggestedPrice: number | null; // Prix estimé en FCFA
  confidence: number;      // 0.0 à 1.0
}
```

4.3 Exemple de réponse Gemini

```
{
  "name": "Alternateur Toyota Corolla",
  "category": "Electricité",
  "oemReference": "27060-0T010",
  "vehicleCompatibility": "Toyota Corolla 2015-2020",
  "suggestedPrice": 45000,
  "confidence": 0.85
}
```

5. Configuration et variables d'environnement

5.1 Variables Gemini

```
GEMINI_API_KEY=AIzaSy...           # Clé API Google Generative AI (obligatoire)
GEMINI_MODEL=gemini-2.0-flash        # Modèle (optionnel, défaut: gemini-2.0-flash)
GEMINI_QUOTA_ALERT_THRESHOLD=0.8     # Seuil d'alerte quota (optionnel, défaut: 0.8)
```

5.2 Variables R2 (stockage images)

```
CLOUDFLARE_R2_ACCESS_KEY_ID=...     # Clé d'accès R2
CLOUDFLARE_R2_SECRET_ACCESS_KEY=...  # Secret R2
R2_BUCKET=pieces-images              # Nom du bucket
R2_PUBLIC_URL=https://images.pieces.ci # URL publique
```

5.3 Comportement sans configuration

Variable manquante	Comportement
GEMINI_API_KEY	Log warning, identification retourne <code>null</code> → saisie manuelle
R2_*	Upload d'images non fonctionnel

6. Endpoint : identification par photo

6.1 Route

POST /api/v1/vision/identify
Auth: Aucune (endpoint public)
Content-Type: multipart/form-data

6.2 Paramètres

Paramètre	Type	Obligatoire	Description
file	File	Oui	Image (JPEG, PNG, WebP, ≤5 Mo)
brand	Query	Non	Filtre marque véhicule
model	Query	Non	Filtre modèle véhicule
year	Query	Non	Filtre année véhicule

6.3 Réponse

```
interface VisionIdentifyResult {  
  status: 'identified' | 'disambiguation' | 'failed';  
  identification: PartIdentification | null;  
  candidates: CatalogCandidate[];  
  matchingParts: CatalogCandidate[];  
}
```

6.4 Exemple de réponse (identifié)

```
{
  "status": "identified",
  "identification": {
    "name": "Alternateur Toyota Corolla",
    "category": "Electricité",
    "oemReference": "27060-0T010",
    "vehicleCompatibility": "Toyota Corolla 2015-2020",
    "suggestedPrice": 45000,
    "confidence": 0.85
  },
  "candidates": [],
  "matchingParts": [
    {
      "id": "catalog-123",
      "name": "Alternateur Toyota Corolla 2017",
      "category": "Electricité",
      "price": 42000,
      "imageThumbUrl": "https://images.pieces.ci/..._thumb.webp",
      "vendor": { "id": "vendor-456", "shopName": "Auto Parts Abidjan" }
    }
  ]
}
```

7. Upload de photo : formats et limites

7.1 Formats acceptés

Format	MIME Type	Extension
JPEG	image/jpeg	.jpg, .jpeg
PNG	image/png	.png
WebP	image/webp	.webp

7.2 Limites

Limite	Valeur
Taille maximale	5 Mo (5 242 880 octets)
Dimensions minimales	300 × 300 px (pour la qualité)

7.3 Configuration Fastify

```
// apps/api/src/server.ts
fastify.register(multipart, {
  limits: { fileSize: 5 * 1024 * 1024 } // 5 Mo
});
```

7.4 Validation dans la route

```
const file = await request.file();

if (!file) return reply.status(400).send({ error: 'Image requise' });

const ALLOWED_TYPES = ['image/jpeg', 'image/png', 'image/webp'];
if (!ALLOWED_TYPES.includes(file.mimetype)) {
  return reply.status(422).send({ error: 'Format invalide (JPG, PNG, WebP)' });
}

const buffer = await file.toBuffer();
if (buffer.length > 5 * 1024 * 1024) {
  return reply.status(422).send({ error: 'Image trop grande (max 5 Mo)' });
}
```

7.5 Conseils photo pour l'utilisateur

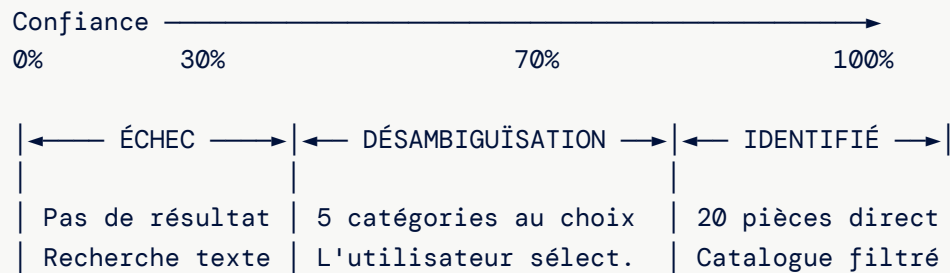
- ▶ Fond neutre (table, sol propre)
- ▶ Bon éclairage (lumière naturelle de préférence)
- ▶ Pièce entière dans le cadre
- ▶ Éviter reflets et ombres marquées

8. Les 3 niveaux de confiance

8.1 Seuils

```
const HIGH_CONFIDENCE_THRESHOLD = 0.7; // 70%
const LOW_CONFIDENCE_THRESHOLD = 0.3; // 30%
```

8.2 Matrice de décision



8.3 Résumé

Confiance	Status	Pièces retournées	Action utilisateur
≥ 70%	identified	Max 20 (matchingParts)	Parcourir les résultats
30-70%	disambiguation	Max 5 (candidates)	Sélectionner une catégorie
< 30%	failed	0	Utiliser la recherche texte

9. Confiance haute (≥ 70%) : identifié

9.1 Requête catalogue

```
// Recherche par nom OU catégorie
where.OR = [
  { name: { contains: identification.name, mode: 'insensitive' } },
  { category: { equals: identification.category, mode: 'insensitive' } },
];

// Filtre optionnel par véhicule
if (vehicleFilter?.brand) {
  where.vehicleCompatibility = {
    contains: vehicleFilter.brand,
    mode: 'insensitive'
  };
}

// Filtres obligatoires
where.status = 'PUBLISHED';
where.inStock = true;
where.vendor = { status: 'ACTIVE' };
```

9.2 Résultats

- ▶ **Maximum** : 20 pièces
- ▶ **Tri** : Plus récentes d'abord (`createdAt desc`)
- ▶ **Contenu** : id, nom, catégorie, prix, miniature, info vendeur

9.3 Exemple

 Photo d'un alternateur → Confiance: 85%

✅ "Alternateur identifié"

Catégorie : Électricité

Référence OEM : 27060-0T010

Compatible : Toyota Corolla 2015-2020

Pièces disponibles (3) :

1. Alternateur Toyota Corolla – 42 000 FCFA – Auto Parts Abidjan
2. Alternateur Toyota 2015-2020 – 38 500 FCFA – Pièces Express
3. Alternateur reconditionné – 35 000 FCFA – Mécanique Plus

10. Confiance moyenne (30-70%) : désambiguïsation

10.1 Requête catalogue

```
// Recherche par catégorie OU premier mot du nom
where.OR = [
  { category: { equals: identification.category, mode: 'insensitive' } },
  { name: { contains: identification.name.split(' ')[0] ?? '', mode:
    'insensitive' } },
];

// Résultats distincts par catégorie
distinct: ['category'];
take: 5;
```

10.2 Résultats

- ▶ **Maximum** : 5 catégories distinctes
- ▶ **Tri** : Plus récentes d'abord
- ▶ **But** : Proposer des catégories pour que l'utilisateur affine

10.3 Exemple

 Photo ambiguë → Confiance: 52%

 "Plusieurs possibilités détectées :"

1. Alternateur – Voir 8 pièces
2. Démarreur – Voir 5 pièces
3. Compresseur clim – Voir 3 pièces
4. Pompe direction – Voir 2 pièces
5. Moteur essuie-glace – Voir 1 pièce

Sélectionnez la bonne catégorie →

11. Confiance basse (< 30%) : échec

11.1 Comportement

- ▶ Aucune pièce retournée
- ▶ L'utilisateur est invité à utiliser d'autres méthodes

11.2 Exemple

 Photo trop floue → Confiance: 15%

 "Identification impossible"

Essayez :

-  Recherche textuelle → Tapez le nom de la pièce
-  Navigation véhicule → Choisissez marque / modèle / année
-  Nouvelle photo → Meilleur éclairage, fond neutre
-  Code VIN → Entrez le VIN de votre véhicule

12. Endpoint : désambiguïsation

12.1 Route

POST /api/v1/vision/disambiguate
Auth: Aucune (endpoint public)
Content-Type: application/json

12.2 Corps de la requête

```
{
  "category": "Electricité",
  "brand": "Toyota"
}
```

Champ	Type	Obligatoire	Description
category	String	Oui	Catégorie sélectionnée par l'utilisateur
brand	String	Non	Filtre optionnel par marque

12.3 Traitement

```
export async function searchByCategory(
  category: string,
  vehicleFilter?: { brand?: string }
) {
  const where = {
    category: { equals: category, mode: 'insensitive' },
    status: 'PUBLISHED',
    inStock: true,
    vendor: { status: 'ACTIVE' },
  };

  if (vehicleFilter?.brand) {
    where.vehicleCompatibility = {
      contains: vehicleFilter.brand,
      mode: 'insensitive'
    };
  }

  return prisma.catalogItem.findMany({
    where,
    take: 20,
    orderBy: { createdAt: 'desc' },
    select: { id, name, category, price, imageThumbUrl, vendor }
  });
}
```

12.4 Réponse

```
{
  "data": [
    {
      "id": "catalog-123",
      "name": "Alternateur Toyota Corolla 2017",
      "category": "Electricité",
      "price": 42000,
      "imageThumbUrl": "https://...",
      "vendor": { "id": "vendor-456", "shopName": "Auto Parts Abidjan" }
    }
  ]
}
```

13. Correspondance catalogue (matching)

13.1 Filtres obligatoires

Tous les résultats doivent satisfaire :

Filtre	Condition
status	PUBLISHED
inStock	true
vendor.status	ACTIVE

13.2 Stratégie de matching

Confiance	Critères de recherche	Résultats
≥ 70%	Nom contient OU catégorie exacte	20 max
30-70%	Catégorie exacte OU premier mot du nom	5 max (distinct catégorie)
< 30%	Aucune recherche	0

13.3 Structure des résultats

```
interface CatalogCandidate {
  id: string;
  name: string | null;
  category: string | null;
  price: number | null;
  imageThumbUrl: string | null;
  vendor: {
    id: string;
    shopName: string;
  };
}
```

14. Décodage VIN (NHTSA VPIC)

14.1 Endpoint

POST /api/v1/browse/vin-decode
Auth: Aucune (endpoint public)
Content-Type: application/json

14.2 Validation

```
const vinDecodeSchema = z.object({
  vin: z.string()
    .length(17, 'Le VIN doit contenir exactement 17 caractères')
    .regex(/^([A-HJ-NPR-Z0-9]){17}$/i, 'Format VIN invalide'),
});
```

Note : Les lettres I, O et Q sont exclues du format VIN (confusion avec 1, 0 et 9).

14.3 Fournisseur : NHTSA VPIC

URL: <https://vpic.nhtsa.dot.gov/api/vehicles/DecodeVinValues/{VIN}?format=json>

Élément	Détail
Fournisseur	National Highway Traffic Safety Administration (USA)
Base de données	Vehicle Product Information Catalog
Couverture	Principalement véhicules nord-américains
Coût	Gratuit, sans clé API
Limitation	Véhicules européens/asiatiques non couverts

14.4 Processus

```
export async function decodeVin(vin: string): Promise<VinDecodeResult> {
  const upperVin = vin.toUpperCase();
  const res = await fetch(
    `https://vpic.nhtsa.dot.gov/api/vehicles/DecodeVinValues/${upperVin}?
      format=json`
  );
  const data = await res.json();
  const result = data.Results?.[0];

  if (!result?.Make) {
    return { vin: upperVin, make: null, model: null, year: null, decoded: false };
  }

  return {
    vin: upperVin,
    make: result.Make,
    model: result.Model,
    year: parseInt(result.ModelYear) || null,
    decoded: true,
  };
}
```

14.5 Réponse

```
{
  "vin": "JTDKN3DU5A0123456",
  "make": "Toyota",
  "model": "Corolla",
  "year": 2017,
  "decoded": true
}
```

14.6 Gestion d'erreurs

Erreur	Résultat
VIN non reconnu	<code>{ decoded: false, make: null }</code>
Erreur réseau	<code>{ decoded: false }</code>
Timeout	<code>{ decoded: false }</code>

15. Recherche textuelle et synonymes

15.1 Endpoint

```
GET /api/v1/browse/search?q={terme}&category={cat}&page=1&limit=20
Auth: Aucune (endpoint public)
```

15.2 Correction des fautes de frappe

Le système utilise une table de **synonymes** pour corriger les erreurs courantes :

```
model SearchSynonym {
  id          String @id @default(uuid())
  typo        String @unique           // La faute
  correction  String                   // La correction
}
```

15.3 Processus de correction

1. Charger tous les synonymes de la base
2. Pour chaque synonyme :
Si la requête contient le typo → remplacer par la correction
3. Exécuter la recherche avec la requête corrigée

Exemple :

```
Saisie utilisateur : "filtre a huile"
Synonyme : typo="huile" → correction="huile"
Requête corrigée : "filtre a huile"
```

15.4 Champs recherchés

La recherche textuelle couvre simultanément :

Champ	Type de recherche
name	Contient (insensible casse)
category	Contient (insensible casse)
oemReference	Contient (insensible casse)
vehicleCompatibility	Contient (insensible casse)

15.5 Réponse paginée

```
{
  "correctedQuery": "filtre a huile",
  "items": [...],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 45,
    "totalPages": 3
  }
}
```

16. Navigation par marque / modèle / année

16.1 Endpoints

GET /api/v1/browse/brands	→ Liste des marques
GET /api/v1/browse/brands/{brand}/models	→ Modèles d'une marque
GET /api/v1/browse/brands/{brand}/models/{model}/years	→ Années d'un modèle
GET /api/v1/browse/parts?brand=X&model=Y&year=Z	→ Pièces compatibles

16.2 Données véhicules

14 marques avec modèles et plages d'années (2000-2024) :

Marque	Exemples de modèles
Toyota	Corolla, Hilux, Camry, RAV4, Land Cruiser
Peugeot	206, 207, 208, 301, 308, 3008
Renault	Clio, Megane, Logan, Duster
Hyundai	i10, i20, Tucson, Santa Fe
Kia	Picanto, Rio, Sportage
Nissan	Almera, Qashqai, X-Trail, Patrol
Mercedes	Classe C, Classe E, Sprinter
Volkswagen	Polo, Golf, Tiguan
Ford	Fiesta, Focus, Ranger
Suzuki	Swift, Vitara, Jimny
Mitsubishi	L200, Pajero, Outlander
Honda	Civic, CR-V, Fit
BMW	Série 3, Série 5, X3
Isuzu	D-Max

16.3 Filtrage par compatibilité

```

if (filters.brand) {
  const compatParts = [filters.brand];
  if (filters.model) compatParts.push(filters.model);
  if (filters.year) compatParts.push(String(filters.year));
  const compatQuery = compatParts.join(' ');

  where.vehicleCompatibility = {
    contains: compatQuery,
    mode: 'insensitive'
  };
}

```

17. Catégories de pièces

17.1 Les 10 catégories

Définies dans le prompt Gemini et dans les constantes :

Catégorie	Exemples
Filtration	Filtre à huile, filtre à air, filtre habitacle
Freinage	Plaquettes, disques, étriers, mâchoires
Suspension	Amortisseurs, ressorts, bras de suspension
Moteur	Alternateur, démarreur, pompe à eau, courroie
Transmission	Embrayage, boîte de vitesses, cardans
Electricité	Batterie, bougies, capteurs, faisceaux
Carrosserie	Pare-chocs, rétroviseurs, phares, capot
Echappement	Pot d'échappement, catalyseur, silencieux
Refroidissement	Radiateur, thermostat, ventilateur
Autre	Tout ce qui ne rentre pas dans les catégories ci-dessus

17.2 Endpoint

```
GET /api/v1/browse/categories
→ Retourne la liste PART_CATEGORIES
```

18. Compatibilité véhicule

18.1 Stockage

Le champ `vehicleCompatibility` sur `CatalogItem` est un **texte libre** :

```
model CatalogItem {
  vehicleCompatibility String? // "Toyota Hilux 2005-2015"
}
```

18.2 Sources de compatibilité

Source	Fiabilité
Gemini (depuis la photo)	Variable (suggérée)
Saisie vendeur	Fiable (connaissance terrain)
VIN decode	Fiable (base NHTSA)

18.3 Filtrage

La compatibilité est filtrée par `contains` (insensible casse) sur la concaténation marque + modèle + année.

18.4 Garage utilisateur

Les utilisateurs peuvent sauvegarder jusqu'à 5 véhicules :

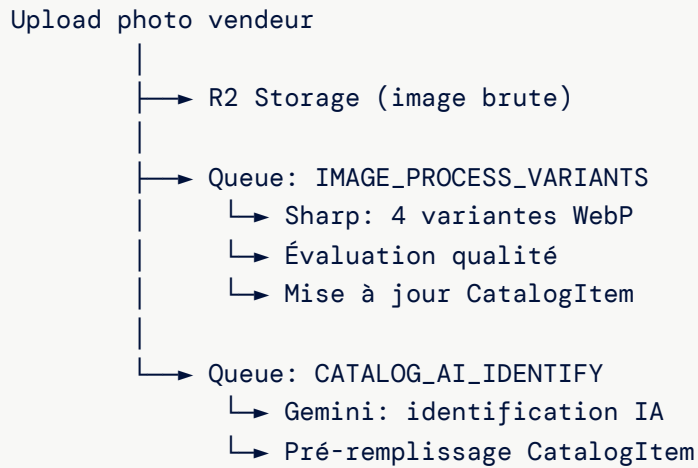
```
model UserVehicle {
  id      String @id @default(uuid())
  userId  String
  brand   String
  model   String
  year    Int
  vin     String?
}
```

Ces véhicules servent de raccourcis pour rechercher des pièces compatibles.

19. Traitement d'images asynchrone (queue)

19.1 Architecture

Quand un vendeur upload une photo, deux jobs sont créés :

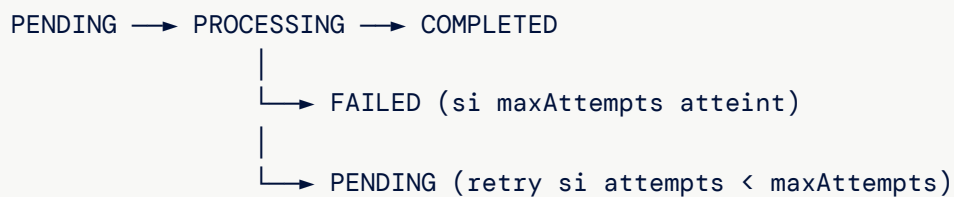


19.2 Worker

```
// Poll toutes les 30 secondes
const POLL_INTERVAL = 30_000;

// Handlers par type de job
const handlers = {
  IMAGE_PROCESS_VARIANTS: handleImageProcess,
  CATALOG_AI_IDENTIFY: handleAiIdentify,
};
```

19.3 Cycle de vie d'un job



État	Description
PENDING	En attente de traitement
PROCESSING	En cours (verrouillé)
COMPLETED	Terminé avec succès
FAILED	Tous les essais épuisés

19.4 Retry

- ▶ **Max attempts**: 3 (par défaut)

- ▶ **Verrouillage:** `SELECT FOR UPDATE SKIP LOCKED` (pas de double traitement)

20. Variantes d'images et qualité

20.1 Les 4 variantes

Chaque image est convertie en **4 tailles WebP** :

Variante	Largeur	Usage
thumb	150 px	Miniatures dans les listes
small	400 px	Résultats de recherche
medium	800 px	Page de détail mobile
large	1200 px	Page de détail desktop

20.2 Traitement Sharp

```
// Bibliothèque : sharp

// Pour chaque variante :
const variant = await sharp(buffer)
  .resize(width, null, {
    fit: 'inside',
    withoutEnlargement: true // Ne pas agrandir les petites images
  })
  .webp({ quality: 80 })
  .toBuffer();
```

20.3 Clés de stockage R2

```
catalog/{vendorId}/{timestamp}_{filename}.jpg      ← Original
catalog/{vendorId}/{timestamp}_{filename}_thumb.webp ← 150px
catalog/{vendorId}/{timestamp}_{filename}_small.webp ← 400px
catalog/{vendorId}/{timestamp}_{filename}_medium.webp ← 800px
catalog/{vendorId}/{timestamp}_{filename}_large.webp ← 1200px
```

20.4 Mise à jour CatalogItem

```
await prisma.catalogItem.update({
  where: { id: catalogItemId },
  data: {
    imageThumbUrl,      // URL publique thumb
    imageSmallUrl,      // URL publique small
    imageMediumUrl,     // URL publique medium
    imageLargeUrl,      // URL publique large
    qualityScore,       // Score 0-1
    qualityIssue,       // Messages d'avertissement
  }
});
```

21. Évaluation de qualité photo

21.1 Fonction

```
function assessQuality(buffer: Buffer): Promise<QualityAssessment> {
  // Retourne : { score: number, issues: string[] }
}
```

21.2 Critères évalués

Critère	Seuil	Message si échec
Dimensions minimales	300 × 300 px	"Image trop petite (minimum 300x300 px)"
Netteté	écart-type < 20	"Image floue — veuillez reprendre la photo"
Luminosité basse	moyenne < 30	"Image trop sombre — activez le flash"
Luminosité haute	moyenne > 240	"Image surexposée — évitez la lumière directe"

21.3 Scoring

Score de base : 1.0
Chaque problème : -0.3
Score minimum : 0.0

Exemples :

- ▶ 0 problème → score 1.0 (excellent)

- ▶ 1 problème → score 0.7 (acceptable)
- ▶ 2 problèmes → score 0.4 (médiocre)
- ▶ 3+ problèmes → score 0.0 (rejet recommandé)

21.4 Analyse technique

La netteté est évaluée via l'**écart-type** des niveaux de gris (Sharp greyscale stats). Une image floue a un écart-type faible (peu de variation de contraste).

22. Job CATALOG_AI_IDENTIFY

22.1 Déclencheur

Créé automatiquement quand un vendeur upload une photo dans le catalogue.

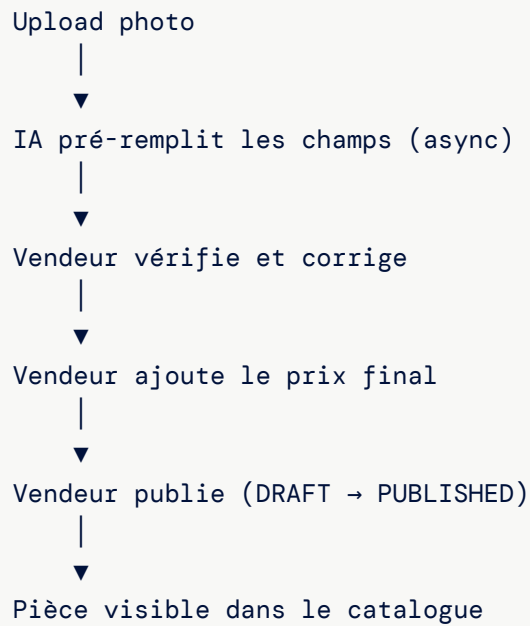
22.2 Processus

1. Télécharger l'image depuis R2
2. Envoyer à Gemini pour identification
3. Si identification réussie :
 - Pré-remplir : name, category, oemReference, vehicleCompatibility, suggestedPrice
 - Marquer : aiConfidence = score, aiGenerated = true
4. Si identification échouée :
 - Marquer : aiGenerated = false
 - Le vendeur doit remplir manuellement

22.3 Champs pré-remplis

```
await prisma.catalogItem.update({
  where: { id: catalogItemId },
  data: {
    name: identification.name,
    category: identification.category,
    oemReference: identification.oemReference,
    vehicleCompatibility: identification.vehicleCompatibility,
    suggestedPrice: identification.suggestedPrice,
    aiConfidence: identification.confidence,
    aiGenerated: true,
  }
});
```

22.4 Workflow vendeur après IA



23. Stockage R2 (Cloudflare)

23.1 Fonctions

```
// Upload
async function uploadToR2(key: string, buffer: Buffer, contentType: string):
    Promise<string>
// Retourne l'URL publique

// Download
async function downloadFromR2(key: string): Promise<Buffer>
```

23.2 Structure des clés

```
catalog/{vendorId}/{timestamp}_{filename}.{ext}
```

23.3 URL publique

```
https://images.pieces.ci/catalog/vendor-123/1709300400000_alternateur.jpg
```

24. Upload catalogue vendeur

24.1 Endpoint

```
POST /api/v1/catalog/items/upload
Auth: Bearer Token (rôle SELLER ou ADMIN)
Content-Type: multipart/form-data
```

24.2 Processus complet

1. Valider le fichier (format, taille)
2. Vérifier le vendeur (statut ACTIVE)
3. Uploader l'image brute vers R2
4. Créer le CatalogItem en statut DRAFT
5. Enqueue IMAGE_PROCESS_VARIANTS
6. Enqueue CATALOG_AI_IDENTIFY
7. Retourner le CatalogItem (id, imageOriginalUrl)

24.3 Statuts du CatalogItem

```
DRAFT → PUBLISHED → ARCHIVED
      |
      └─ Stock toggle (inStock: true/false)
```

24.4 Prérequis pour publier

Champ	Obligatoire	Source
name	Oui	IA ou vendeur
category	Oui	IA ou vendeur
price	Oui	Vendeur uniquement

25. Détection bait-and-switch

25.1 Principe

Le système détecte les changements de prix suspects : un vendeur qui augmente significativement le prix peu de temps après la publication.

25.2 Seuils

Critère	Seuil
Variation de prix	> 50%
Fenêtre temporelle	< 1 heure

25.3 Action

```
// Si le vendeur modifie le prix de >50% en <1h
logger.warn({
  catalogItemId,
  oldPrice,
  newPrice,
  changePercent
}, 'PRICE_ALERT_BAIT_SWITCH');
```

En v1, un **warning est loggé** mais aucune action automatique n'est prise. Le monitoring admin peut détecter ces alertes.

26. Intégration WhatsApp (photo par message)

26.1 Réception de photo

Quand un utilisateur envoie une photo via WhatsApp :

```
{
  "type": "image",
  "image": {
    "id": "media_id_from_meta",
    "mime_type": "image/jpeg"
  }
}
```

26.2 Réponse actuelle (v1)

```
"Photo reçue! Identification en cours...
Vous recevrez les résultats sous peu."
```

26.3 Flux planifié (Phase 2)



27. Rate limiting et quotas

27.1 Rate limiting API

Scope	Limite
Global (toutes routes)	100 req/min par IP
Vision identify	Soumis au global (100/min)
Vision disambiguate	Soumis au global (100/min)
Browse search	Soumis au global (100/min)
VIN decode	Soumis au global (100/min)

27.2 Quotas Gemini

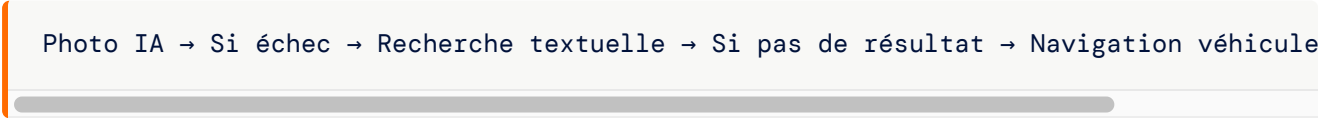
Élément	Valeur
Estimation interne	~100 appels
Seuil d'alerte	80% (configurable)
Action à l'alerte	Log <code>GEMINI_QUOTA_ALERT</code>
Pas de blocage	L'alerte est informative

27.3 Limites NHTSA VPIC

Élément	Valeur
Limite	Non documentée (API gratuite)
Recommandation	Éviter les rafales massives

28. Gestion d'erreurs et fallbacks

28.1 Cascade de fallbacks



28.2 Erreurs vision

Code	HTTP	Message	Fallback
MISSING_IMAGE	400	Image requise	Sélectionner un fichier
INVALID_IMAGE_TYPE	422	Format invalide	JPG, PNG ou WebP
IMAGE_TOO_LARGE	422	Image trop grande	Réduire à < 5 Mo
MISSING_CATEGORY	400	Catégorie requise	Sélectionner une catégorie
Gemini échoue	—	Retourne <code>null</code>	<code>status: 'failed'</code>

28.3 Erreurs catalogue

Code	HTTP	Message
VENDOR_NOT_FOUND	404	Vendeur introuvable
VENDOR_NOT_ACTIVE	403	Vendeur inactif
FILE_TOO_LARGE	422	Fichier trop grand
INVALID_FILE_TYPE	422	Format de fichier invalide
CATALOG_ITEM_NOT_DRAFT	422	L'article n'est pas en brouillon
CATALOG_PRICE_REQUIRED	422	Prix requis pour publier

28.4 Niveaux de log

Niveau	Événements
INFO	CATALOG_IMAGE_UPLOADED, IMAGE_VARIANTS_PROCESSED, CATALOG_AI_IDENTIFIED
WARN	GEMINI_NOT_CONFIGURED, GEMINI_QUOTA_ALERT, GEMINI_API_FAILED, CATALOG_AI_FALLBACK, PRICE_ALERT_BAIT_SWITCH

29. Tests unitaires

29.1 Tests vision service

Fichier : `apps/api/src/modules/vision/vision.service.test.ts`

Test	Description
Confiance haute	Retourne <code>identified</code> + matchingParts
Confiance basse	Retourne <code>disambiguation</code> + candidates
Gemini null	Retourne <code>failed</code>
searchByCategory	Retourne les pièces de la catégorie

29.2 Tests vision routes

Fichier : `apps/api/src/modules/vision/vision.routes.test.ts`

Test	Description
Upload valide	Identifie et retourne résultat
Format invalide	422 error
Taille invalide	422 error
Fichier manquant	400 error
Disambiguate valide	Retourne les pièces
Catégorie manquante	400 error

29.3 Tests browse

Fichier : `apps/api/src/modules/browse/browse.service.test.ts`

Test	Description
Search avec synonymes	Corrige les fautes de frappe
Browse par véhicule	Filtre par compatibilité
VIN decode valide	Retourne marque/modèle/année
VIN decode invalide	Retourne <code>decoded: false</code>
Pagination	Page, limit, total, totalPages

29.4 Tests catalogue

Test	Description
Upload image	Crée item DRAFT + enqueue jobs
Publish	Vérifie name + category + price
Stock toggle	inStock true/false
AI fallback	aiGenerated = false si Gemini échoue

30. Modèles de données Prisma

30.1 CatalogItem

```
model CatalogItem {
  id                String      @id @default(uuid())
  vendorId          String
  vendor            Vendor      @relation(fields: [vendorId])
  name              String?
  category           String?
  price             Int?
  oemReference       String?
  vehicleCompatibility String?
  description        String?
  status             CatalogStatus @default(DRAFT)
  inStock            Boolean     @default(true)

  // Images
  imageOriginalUrl   String?
  imageThumbUrl      String?
  imageSmallUrl      String?
  imageMediumUrl     String?
  imageLargeUrl      String?

  // AI
  aiGenerated        Boolean?
  aiConfidence        Float?
  suggestedPrice      Int?
  qualityScore        Float?
  qualityIssue        String?

  createdAt          DateTime    @default(now())
  updatedAt           DateTime    @updatedAt
}
```

30.2 SearchSynonym

```
model SearchSynonym {
  id      String @id @default(uuid())
  typo    String @unique
  correction String
}
```

30.3 UserVehicle

```
model UserVehicle {
  id      String @id @default(uuid())
  userId  String
  user    User   @relation(fields: [userId])
  brand   String
  model   String
  year    Int
  vin     String?
}
```

30.4 Job (queue)

```
model Job {
  id          String      @id @default(uuid())
  type        JobType
  payload     Json
  status      JobStatus   @default(PENDING)
  attempts    Int         @default(0)
  maxAttempts Int         @default(3)
  scheduledAt DateTime?
  completedAt DateTime?
  createdAt   DateTime    @default(now())
}

enum JobType {
  IMAGE_PROCESS_VARIANTS
  CATALOG_AI_IDENTIFY
}

enum JobStatus {
  PENDING
  PROCESSING
  COMPLETED
  FAILED
}
```

30.5 Enums

```
enum CatalogStatus {
  DRAFT
  PUBLISHED
  ARCHIVED
}
```

31. Validation des données (Zod)

31.1 VIN

```
export const vinDecodeSchema = z.object({
  vin: z.string()
    .length(17, 'Le VIN doit contenir exactement 17 caractères')
    .regex(/^([A-HJ-NPR-Z0-9]){17}$/i, 'Format VIN invalide'),
});
```

31.2 Recherche

```
export const searchSchema = z.object({
  q: z.string().min(2, 'Au moins 2 caractères'),
  category: z.string().optional(),
  page: z.coerce.number().int().min(1).optional(),
  limit: z.coerce.number().int().min(1).max(100).optional(),
});
```

31.3 Désambiguïsation

```
export const disambiguateSchema = z.object({
  category: z.string().min(1, 'Catégorie requise'),
  brand: z.string().optional(),
});
```

32. État de l'implémentation

32.1 Implémenté (v1)

Composant	État	Fichier
Identification photo (Gemini)	Implémenté	<code>vision.service.ts</code>
Upload multipart (JPEG/PNG/WebP)	Implémenté	<code>vision.routes.ts</code>
3 niveaux de confiance	Implémentés	<code>vision.service.ts</code>
Désambiguïsation	Implémentée	<code>vision.routes.ts</code>
Matching catalogue	Implémenté	<code>vision.service.ts</code>
Décodage VIN (NHTSA)	Implémenté	<code>browse.service.ts</code>
Recherche textuelle + synonymes	Implémentée	<code>browse.service.ts</code>
Navigation marque/modèle/année	Implémentée	<code>browse.routes.ts</code>
Catégories de pièces	Implémentées	Constantes
Traitement images async (Sharp)	Implémenté	<code>imageProcess.ts</code>
4 variantes WebP	Implémentées	<code>imageProcess.ts</code>
Évaluation qualité photo	Implémentée	<code>imageProcessor.ts</code>
Job CATALOG_AI_IDENTIFY	Implémenté	<code>handlers/</code>
Stockage R2 (Cloudflare)	Implémenté	<code>lib/r2.ts</code>
Upload catalogue vendeur	Implémenté	<code>catalog.service.ts</code>
Détection bait-and-switch	Implémentée	<code>catalog.service.ts</code>
Tests unitaires	Implémentés	<code>*.test.ts</code>

32.2 Non implémenté (planifié)

Composant	Priorité	Description
Identification via WhatsApp	Haute	Télécharger image Meta → Gemini
Recherche similaire par image	Moyenne	Comparer avec images du catalogue
Cache des résultats Gemini	Moyenne	Éviter les appels redondants
OCR référence OEM	Basse	Lire la référence imprimée sur la pièce
Apprentissage continu	Basse	Améliorer les résultats avec les retours utilisateurs

33. Référence des fichiers

33.1 Module vision

Fichier	Chemin	Rôle
Routes	<code>apps/api/src/modules/vision/vision.routes.ts</code>	Endpoints identify + disambiguate
Service	<code>apps/api/src/modules/vision/vision.service.ts</code>	identifyFromPhoto, searchByCategory
Tests routes	<code>apps/api/src/modules/vision/vision.routes.test.ts</code>	Tests endpoints
Tests service	<code>apps/api/src/modules/vision/vision.service.test.ts</code>	Tests logique

33.2 Module browse

Fichier	Chemin	Rôle
Routes	<code>apps/api/src/modules/browse/browse.routes.ts</code>	Search, brands, VIN
Service	<code>apps/api/src/modules/browse/browse.service.ts</code>	searchParts, decodeVin
Tests	<code>apps/api/src/modules/browse/browse.service.test.ts</code>	Tests recherche

33.3 Module catalogue

Fichier	Chemin	Rôle
Routes	<code>apps/api/src/modules/catalog/catalog.routes.ts</code>	Upload, edit, publish
Service	<code>apps/api/src/modules/catalog/catalog.service.ts</code>	CRUD catalogue
Tests	<code>apps/api/src/modules/catalog/catalog.service.test.ts</code>	Tests catalogue

33.4 Librairies

Fichier	Chemin	Rôle
Gemini	<code>apps/api/src/lib/gemini.ts</code>	Google Generative AI
Image processor	<code>apps/api/src/lib/imageProcessor.ts</code>	Sharp + qualité
R2 storage	<code>apps/api/src/lib/r2.ts</code>	Cloudflare R2

33.5 Queue

Fichier	Chemin	Rôle
Queue service	<code>apps/api/src/modules/queue/queueService.ts</code>	enqueue, dequeue
Worker	<code>apps/api/src/modules/queue/worker.ts</code>	Poll + dispatch
Image handler	<code>apps/api/src/modules/queue/handlers/imageProcess.ts</code>	Variantes + qualité
AI handler	<code>apps/api/src/modules/queue/handlers/</code>	Identification Gemini

33.6 Constantes et validateurs

Fichier	Chemin	Contenu
Véhicules	<code>packages/shared/constants/vehicles.ts</code>	14 marques, modèles, années
Catégories	<code>packages/shared/constants/</code>	PART_CATEGORIES
Validators	<code>packages/shared/validators/browse.ts</code>	vinDecodeSchema, searchSchema
Schema	<code>packages/shared/prisma/schema.prisma</code>	CatalogItem, SearchSynonym, Job

34. FAQ technique

Q : L'identification IA fonctionne-t-elle sans clé Gemini ?

R : Non. Sans `GEMINI_API_KEY`, la fonction `identifyPart()` retourne `null`, ce qui donne le status `failed`. L'utilisateur devra utiliser la recherche textuelle ou la navigation par véhicule.

Q : Quels modèles Gemini sont supportés ?

R : Le modèle par défaut est `gemini-2.0-flash` (rapide et économique). Vous pouvez changer via la variable `GEMINI_MODEL`. Tout modèle Google Generative AI supportant les images est compatible.

Q : La recherche supporte-t-elle la recherche floue (fuzzy) ?

R : Partiellement. Le système utilise des **synonymes manuels** (table `SearchSynonym`) pour corriger les fautes courantes. La recherche utilise `contains` (sous-chaîne) plutôt qu'une recherche floue complète. PostgreSQL `pg_trgm` est disponible mais pas encore exploité pour le scoring.

Q : Comment ajouter un nouveau synonyme de recherche ?

R : Insérez directement dans la base :

```
INSERT INTO search_synonyms (id, typo, correction)
VALUES (gen_random_uuid(), 'alternature', 'alternateur');
```

Q : L'évaluation de qualité bloque-t-elle l'upload ?

R : Non. L'évaluation est **informative** — elle stocke un score et des messages d'avertissement mais ne bloque pas. Le vendeur peut publier même avec un score bas. L'interface pourrait afficher les avertissements pour encourager une meilleure photo.

Q : Combien de temps prend l'identification IA ?

R : L'appel Gemini prend généralement **2-5 secondes**. Le traitement des variantes d'images (Sharp) prend **1-3 secondes** supplémentaires. Ces deux processus sont exécutés en parallèle dans la queue.

Q : Le VIN decode fonctionne-t-il pour les véhicules européens ?

R : Partiellement. La base NHTSA VPIC couvre principalement les véhicules vendus en Amérique du Nord. Les véhicules européens ou asiatiques avec un VIN standard peuvent être reconnus, mais la couverture n'est pas garantie. En cas d'échec, l'utilisateur est redirigé vers la navigation manuelle par marque/modèle/année.

Q : Comment fonctionne le pré-remplissage IA pour les vendeurs ?

R : Quand un vendeur upload une photo, le job `CATALOG_AI_IDENTIFY` appelle Gemini et pré-remplit : nom, catégorie, référence OEM, compatibilité véhicule, prix suggéré. Le vendeur peut ensuite corriger ces champs avant de publier. L'article reste en DRAFT jusqu'à la publication manuelle.

Q : Le système détecte-t-il les fausses photos ?

R : Pas explicitement. L'évaluation de qualité vérifie les dimensions, la netteté et la luminosité, mais ne détecte pas les photos copiées d'internet ou les images trompeuses. La détection de fraude photo est planifiée pour une version future.

Q : Peut-on utiliser l'API Vision sans interface web ?

R : Oui. L'endpoint `POST /api/v1/vision/identify` est **public** (pas d'authentification). Vous pouvez l'appeler directement avec un `multipart/form-data` contenant le champ `file`. Idéal pour les intégrations tierces ou le bot WhatsApp.